

BÁO CÁO ĐỒ ÁN MÔN HỌC

(Đồ án phát triển ứng dụng)

Lớp: IT003.Q21.CTTN

SINH VIÊN THỰC HIỆN: Lê Đức Nhân

Mã sinh viên: 25521274

Họ và tên: Lê Đức Nhân

TÊN ĐỀ TÀI: Xây dựng ứng dụng mô phỏng trò chơi bóng đá 2D tích hợp bot ra quyết định sử dụng các giải thuật tìm chặn, điều hướng và mô phỏng vật lý

1. Giới thiệu đồ án:

a. Mô tả chung về ứng dụng:

Ứng dụng mô phỏng trò chơi bóng đá 2D theo thời gian thực, gồm:

- mô phỏng chuyển động cầu thủ và bóng
- xử lý va chạm vật lý
- cơ chế sút bóng và ghi bàn
- bot AI điều khiển cầu thủ máy
- các kỹ năng đặc biệt cho nhân vật

Mục tiêu:

- xây dựng game mô phỏng
- áp dụng các kiến thức Cấu trúc dữ liệu và Giải thuật
- phát triển hệ thống bot có khả năng ra quyết định.

b. Các CTDL và giải thuật đã được sử dụng:

i. Vector (pygame.Vector2):

- Dùng biểu diễn vị trí, vận tốc, hướng di chuyển và hướng sút của cầu thủ, bóng.
- Áp dụng các phép toán vector như `normalize()` để xác định hướng di chuyển.

- Sử dụng dot() để kiểm tra căn chỉnh góc sút, hướng tiếp cận bóng và định hướng tương đối giữa bot, bóng và khung thành
 - Dùng trong tính toán phản xạ va chạm giữa bóng với cột và tường
 - Hỗ trợ tính toán hướng sút có bù vận tốc bóng, giúp bot sút chính xác hơn khi bóng đang chuyển động
- ii. Danh sách (List)
- Để lưu: goal posts
- iii. Thuật toán tính đường đi ngắn nhất đến quả bóng khi bóng đang di chuyển
- Mô phỏng quỹ đạo tương lai của bóng, bot không chạy tới vị trí hiện tại của bóng mà tới nơi bóng sẽ xuất hiện.
- iv. Thuật toán ra quyết định
- Bot gồm 4 trạng thái: ATTACK, DEFEND, CLEAR, SPAM -> chia bài toán quyết định thành các trạng thái
- v. Hàng đợi vòng (Circular Queue - deque)
- Sử dụng để lưu lịch sử trạng thái trận đấu theo từng frame
 - Giới hạn số frame để tránh tràn bộ nhớ
 - Dùng cho hệ thống replay và tua ngược thời gian

2. Quá trình thực hiện

a. Tuần 1:

- Xây dựng sân bóng và rendering.
- Cài đặt vật lý bóng.
- Cài đặt di chuyển cầu thủ.
- Xử lý va chạm cơ bản.
- Hoàn thiện cơ chế sút và ghi bàn.

b. Tuần 2:

- Xây dựng bot AI cơ bản.
- Cài đặt thuật toán tính đường đi ngắn nhất để bot di chuyển tối ưu.
- Phát triển các trạng thái của bot.

- Tối ưu góc sút.
- Thêm debug system để kiểm tra vector và trạng thái bot.

c. Tuần 3:

- Xây dựng hệ thống Replay khi ghi bàn
 - (1) Lưu trạng thái trận đấu theo từng frame vào cấu trúc hàng đợi vòng
 - (2) Khi có bàn thắng, phát lại các frame trước đó để hiển thị pha bóng
 - (3) Áp dụng trong vòng lặp game bằng cách chuyển sang chế độ "REPLAY"
- Phát triển kỹ năng quay ngược thời gian (Rewind) cho nhân vật
 - (1) Lưu snapshot toàn bộ trạng thái trận đấu (bóng, cầu thủ, hiệu ứng)
 - (2) Khi kích hoạt kỹ năng, hệ thống lần lượt khôi phục lại các trạng thái trước đó
 - (3) Sử dụng deque để lưu lịch sử trạng thái (rewind_history)
- Xây dựng cơ chế freeze và hiệu ứng sau khi rewind.
 - (1) Sau khi tua ngược, trận đấu tạm dừng trong một khoảng thời gian ngắn
 - (2) Kết hợp hiệu ứng hình ảnh để thể hiện việc “ổn định lại thời gian”
- Tích hợp replay với hệ thống hiệu ứng.
 - (1) Lưu và khôi phục trạng thái
 - (2) Đảm bảo replay hiển thị đúng toàn bộ diễn biến

3. Kết quả đạt được

- game chạy thời gian thực
- xử lý va chạm
- bot AI có thể: phòng thủ, tranh bóng, sút ghi bàn

4. Phụ lục : Giới thiệu (demo) kết quả



Recording

2026-04-21 232844.m

5. Docstring

[ball.py]

class Ball: Đối tượng quả bóng, gồm di chuyển, va chạm và kiểm tra bàn thắng.

def __init__: Đặt bóng ở giữa sân với vận tốc ban đầu bằng 0.
def update: Cập nhật chuyển động của bóng trong một frame.
def handle_wall_collision: Xử lý va chạm tường và trả về sự kiện ghi bàn nếu bóng vào gôn.
def handle_post_collision: Xử lý va chạm giữa bóng và bốn cột gôn.
def reset: Đưa bóng về giữa sân và dừng bóng lại.

[debug.py]

def _vector_angle_deg: Trả về góc theo độ của một vector khác 0 để hiển thị trên màn hình.
class DebugOverlay: Bảng debug trong game để xem trạng thái AI, vector và phím điều khiển.
def __init__: Khởi tạo các tùy chọn hiển thị debug và font tạo khi cần.
def handle_event: Xử lý phím tắt debug và trả về True nếu event đã được dùng.
def draw: Vẽ các lớp debug đang bật lên trên màn hình trận đấu.
def _ensure_fonts: Tạo font debug trong lần đầu overlay được vẽ.
def _draw_pitch_guides: Vẽ các đường hướng dẫn vector quanh bóng và cầu thủ.
def _draw_player_vectors: Vẽ vector mục tiêu, chặn bóng, hướng sút và đường nối bóng của cầu thủ.
def _draw_panel: Vẽ bảng thông tin ngắn về hai cầu thủ và bóng.
def _build_player_line: Tạo một dòng thông tin debug đã định dạng cho cầu thủ.
def _draw_help: Vẽ gợi ý phím tắt debug ở cạnh dưới sân.
def _get_shot_direction: Trả về hướng sút hiện tại, nếu thiếu thì dùng hướng từ cầu thủ tới bóng.
def _screen_pos: Đổi tọa độ trong sân sang tọa độ nguyên trên màn hình.

[effect.py]

def _ease_out_cubic: Làm mượt giá trị chuẩn hóa theo kiểu nhanh lúc đầu và chậm dần về cuối.

class Shockwave: Hiệu ứng vòng tròn lan rộng khi một số kỹ năng được kích hoạt.
def update: Cập nhật shockwave và trả về True nếu hiệu ứng còn tồn tại.
def draw: Vẽ shockwave dưới dạng vòng tròn lan rộng và mờ dần.

class ConvergeCircle: Hiệu ứng vòng tròn hội tụ sau khi kỹ năng quay ngược của Itachi kết thúc.
def update: Cập nhật hiệu ứng hội tụ và trả về True nếu nó còn hiển thị.
def draw: Vẽ vòng tròn mờ dần và co lại về vị trí mục tiêu.

class SkillEffectManager: Quản lý toàn bộ hiệu ứng tạm thời từ cú sút và kỹ năng nhân vật.
def __init__: Khởi tạo danh sách hiệu ứng rỗng và trạng thái overlay.
def reset: Xóa mọi particle, sóng, overlay và trạng thái kỹ năng đang lưu.
def get_state: Chụp snapshot deep-copy để phát lại replay hoặc quay ngược thời gian.
def set_state: Khôi phục snapshot hiệu ứng đã được chụp trước đó.
def set_itachi_overlay: Làm mờ overlay tối của Itachi theo hướng hiện hoặc ẩn.
def start_itachi_converge: Tạo vòng tròn hội tụ sau rewind tại vị trí của Itachi.
def update_freeze_effects: Chỉ cập nhật hiệu ứng ở pha đứng hình khi gameplay đang tạm dừng.
def update: Cập nhật particle kỹ năng, tia sút, shockwave và overlay trong một frame.
def draw: Vẽ overlay trước, sau đó đến particle, shockwave và vòng hội tụ.
def _on_skill_activated: Phát hiệu ứng bùng nổ một lần khi kỹ năng vừa được kích hoạt.
def _emit_nagi_particles: Phát particle bay quanh Nagi liên tục khi kỹ năng đang bật.
def _burst_nagi_particles: Phát hiệu ứng bùng nổ ban đầu của Nagi.
def _emit_bachira_particles: Phát particle của Bachira quanh bóng khi kỹ năng đang bật.
def _burst_bachira_particles: Phát hiệu ứng bùng nổ ban đầu của Bachira.

def _emit_kunigami_particles: Phát vật lửa của Kunigami khi kỹ năng đang bật.
def _burst_kunigami_particles: Phát hiệu ứng bùng nổ ban đầu của Kunigami.
def _emit_chigiri_particles: Phát vật tốc độ của Chigiri khi kỹ năng đang bật.
def _burst_chigiri_particles: Phát hiệu ứng tăng tốc ban đầu của Chigiri.
def _spawn_nagi_particle: Tạo một particle hình lưỡi liềm của Nagi bay về phía cầu thủ.
def _spawn_bachira_particle: Tạo một particle hình vuông của Bachira gần cầu thủ hoặc bóng.
def _spawn_kunigami_particle: Tạo một particle tia lửa của Kunigami.
def _spawn_chigiri_particle: Tạo một vật tốc độ của Chigiri phía sau cầu thủ.
def _emit_itachi_particles: Phát particle tối quanh người dùng kỹ năng quay ngược của Itachi.
def _emit_kick_particles: Phát tia sáng nhỏ theo hướng của cú sút.

[main.py]

def player_state: Chụp trạng thái có thể thay đổi của cầu thủ để dùng cho rewind.
def restore_player_state: Khôi phục trạng thái cầu thủ từ snapshot rewind.
def game_snapshot: Chụp toàn bộ trạng thái trận đấu cho kỹ năng quay ngược của Itachi.
def restore_game_snapshot: Khôi phục bóng, cầu thủ, hiệu ứng và trạng thái hiển thị bóng từ snapshot.
def finish_itachi_rewind: Kết thúc rewind của Itachi và đặt hồi chiêu cho người dùng kỹ năng.
def update_game: Cập nhật một frame gameplay thật và trả về điểm số cùng trạng thái bóng.
def draw_menu: Vẽ menu đầu tiên để chọn chơi với bot hoặc PvP.
def draw_character_select: Vẽ màn hình chọn nhân vật cho người chơi hiện tại.
def main: Chạy vòng lặp pygame gồm menu, gameplay, replay và rewind.

[particle.py]

def _lerp: Nội suy tuyến tính giữa hai giá trị số.

def _smoothstep: Làm mượt giá trị chuẩn hóa để particle đổi kích thước mềm hơn.

class Particle: Particle tròn cơ bản có vận tốc, mờ dần, lực cản và tùy chọn bám mục tiêu.

def update: Cập nhật vật lý particle và trả về True nếu nó còn tồn tại.

def current_size: Trả về kích thước particle tại tiến độ vòng đời hiện tại.

def current_alpha: Trả về độ trong suốt đã mờ dần theo tiến độ vòng đời.

def progress: Trả về tiến độ vòng đời được giới hạn trong khoảng 0..1.

def draw: Vẽ particle dưới dạng hình tròn mờ trong tọa độ màn hình.

class CrescentParticle: Particle được vẽ dưới dạng hình lưỡi liềm.

def draw: Vẽ particle thành lưỡi liềm bằng cách cắt từ hình tròn đặc.

class SquareParticle: Particle được vẽ dưới dạng hình vuông xoay.

def update: Cập nhật vật lý cơ bản và xoay hình vuông.

def draw: Vẽ particle dưới dạng hình vuông xoay và mờ dần.

class ParticleSystem: Bộ chứa quản lý và cập nhật tất cả particle đang hoạt động.

def __init__: Tạo danh sách particle rỗng.

def emit: Thêm một particle vào hệ thống đang hoạt động.

def clear: Xóa toàn bộ particle đang hoạt động.

def update: Cập nhật tất cả particle và loại bỏ particle đã hết hạn.

def draw: Vẽ tất cả particle đang hoạt động với cùng offset màn hình.

[player.py]

class Player: Cầu thủ do người chơi hoặc bot điều khiển, gồm di chuyển, va chạm, AI và kỹ năng.

def __init__: Tạo cầu thủ tại vị trí spawn với màu và bộ phím điều khiển.

def _reset_debug_info: Đặt lại thông tin debug từng frame dùng cho debug overlay.

def simulate_ball_step: Mô phỏng một bước vật lý bóng đơn giản để AI dự đoán.

def estimate_time: Ước lượng thời gian cầu thủ cần để đến một điểm mục tiêu.

def find_intercept_info: Dự đoán vị trí bóng trong tương lai mà cầu thủ có thể chạm tới tốt nhất.

def find_intercept: Trả về riêng vị trí dự đoán để cầu thủ chặn bóng.

def jockey_position: Chọn vị trí phòng thủ giữa đối thủ và gôn của cầu thủ này.

def find_best_clear: Chọn mục tiêu phá bóng an toàn và dễ tiếp cận hơn cho bot.

def is_player_ball_line_on_goal: Kiểm tra cầu thủ có thể sút bóng về phía gôn trái hay không.

def is_not_player_ball_line_on_goal: Kiểm tra cầu thủ có thể phá bóng khỏi vùng gôn phải hay không.

def find_best_goal_target: Chọn điểm hợp lệ tốt nhất trong miệng gôn để sút.

def orbit_attack_target: Dịch mục tiêu tiếp cận vòng quanh bóng khi đường vào trực tiếp không tốt.

def NEED_SPAM: Trả về True nếu bot nên áp sát và spam tranh bóng ở cự ly gần.

def bot_update: Cập nhật di chuyển, chọn mục tiêu và quyết định sút của bot trong một frame.

def handle_input: Áp dụng input bàn phím vào vận tốc của cầu thủ.

def update: Cập nhật vật lý cầu thủ, bộ đếm thời gian, kỹ năng và debug.

def handle_player_collision: Xử lý va chạm vật lý giữa cầu thủ này và cầu thủ khác.

def handle_wall_collision: Giữ cầu thủ trong vùng chơi và trả về phản hiệu chỉnh vị trí.

def handle_ball_collision: Xử lý va chạm giữa cầu thủ này và bóng.

def kick: Sút bóng nếu bóng trong tầm và cầu thủ được phép sút.

def activate_skill: Kích hoạt kỹ năng nhân vật nếu kỹ năng không trong thời gian hồi chiêu.

def information: Gán nhân vật đã chọn và màu hiển thị cho cầu thủ.

def draw: Vẽ cầu thủ dưới dạng hình tròn trong tọa độ sân.

def reset: Đưa cầu thủ về spawn và xóa trạng thái di chuyển, sút, kỹ năng.

[render.py]

def draw_scene: Vẽ toàn bộ trận đấu gồm sân, cầu thủ, bóng, điểm số, hiệu ứng và debug.

[skill_replay.py]

class ReplayBuffer: Bộ nhớ vòng lưu các frame gần nhất để phát lại sau bàn thắng.

def __init__: Tạo replay buffer rỗng với số frame tối đa cố định.

def save: Lưu một frame gồm trạng thái bóng, cầu thủ và hiệu ứng nếu có.

def get_frames: Trả về bản sao dạng list của các frame đã lưu.

def clear: Xóa toàn bộ frame đang lưu trong replay buffer.